

2 Description Détaillée du Modèle de Détection et Protocole d'Entraînement

Ce chapitre expose de manière approfondie et rigoureuse la modélisation mathématique, l'architecture algorithmique sous-jacente, ainsi que l'ensemble des choix techniques et des hyperparamètres retenus pour l'apprentissage du réseau de neurones. Conformément aux exigences de reproductibilité scientifique, les éléments présentés ici fournissent toutes les spécifications nécessaires à la réplique exacte de nos travaux.

2.1 Formalisation Théorique et Discrétisation Spatiale

La problématique posée relève de la détection d'objets multiclasse en boîte unique par cellule. Le but est d'identifier, de localiser par une boîte englobante (*bounding box*) et de classer simultanément un objet appartenant à l'une des $N_c = 9$ classes de balles et ballons de sport définies par l'ensemble :

$$\mathcal{C} = \{\text{'americanFootball'}, \text{'ball_pingpong'}, \text{'ball_tennis'}, \dots, \text{'basketball'}\}$$

Sur le plan conceptuel, le problème est formalisé via une discrétisation de l'espace image sous forme d'une grille bidimensionnelle de taille $S \times S$, où S représente le nombre de cellules par dimension. Les contraintes structurelles initiales imposées par le problème sont régies par les variables suivantes :

- **IMAGE_SIZE** = 64 : Dimension spatiale (largeur et hauteur en pixels) des images d'entrée théoriques.
- **CELL_PER_DIM** = 8 : Nombre de cellules le long de chaque axe, définissant une grille de $8 \times 8 = 64$ cellules.
- **BOX_PER_CELL** = 1 : Nombre maximal de boîtes englobantes prédites par chaque cellule individuelle.
- **NB_CLASSES** = 9 : Dimension spatiale du vecteur de probabilités conditionnelles de classe.
- **PIX_PER_CELL** = $\text{round}(64/8) = 8$: Résolution intrinsèque d'une cellule (zone carrée de 8×8 pixels).

Table 1: Synthèse des paramètres de la structure de données et de modélisation

Paramètre du projet	Variable Code	Valeur	Unité / Signification
Taille de l'image source	IMAGE_SIZE	64	Pixels (Largeur $\times$ Hauteur)
Division de la grille	CELL_PER_DIM	8	Cellules par dimension (S)
Densité de prédiction	BOX_PER_CELL	1	Boîte par cellule (B)
Nombre de cibles	NB_CLASSES	9	Classes distinctes (N_c)
Résolution de cellule	PIX_PER_CELL	8	Pixels par bord de cellule

Chaque cellule d'indice (i, j) de la grille (où $1 \leq i, j \leq S$) a la charge exclusive de prédire un vecteur de sortie théorique $\mathbf{y}_{ij}$ structuré de la manière suivante :

$$\mathbf{y}_{ij} = [C_{ij}, x_{ij}, y_{ij}, w_{ij}, h_{ij}, P_{ij}(1), P_{ij}(2), \dots, P_{ij}(N_c)]^T \quad (1)$$

Où $C_{ij} \in [0, 1]$ désigne le score de confiance indiquant la probabilité qu'un objet ait son centre géométrique à l'intérieur de la cellule (i, j) . Les variables x_{ij} et y_{ij} représentent les coordonnées du centre de la boîte, exprimées relativement aux frontières de la cellule. Les variables w_{ij} et h_{ij} désignent la largeur et la hauteur de la boîte relatives aux dimensions totales de l'image. Enfin, $P_{ij}(k) = P(\text{Classe}_k \mid \text{Objet})$ modélise la distribution de probabilité sur les classes.

2.2 Architecture Détaillée du Réseau : YOLO26n

L'architecture **YOLO26n** (YOLO Nano, version 2026) est optimisée pour le compromis entre l'efficacité computationnelle et le pouvoir d'abstraction sémantique. Elle s'articule autour de trois composants structurels majeurs : le *Backbone*, le *Neck*, et la *Head*.

2.2.1 Le Backbone (Extracteur de Caractéristiques)

Le réseau de base est une variante profonde de DarkNet, caractérisée par l'utilisation de convolutions bidimensionnelles couplées à des fonctions d'activation non linéaires de type **SiLU** et une normalisation par lots (*Batch Normalization*).

Le mécanisme s'appuie sur des blocs **C2f** (*CSPLayer with 2 convolutions*), scindant les cartes de caractéristiques en deux flux pour maximiser le gradient tout en réduisant la redondance des paramètres. En fin de Backbone, un bloc **SPPF** (*Spatial Pyramid Pooling Fast*) applique des opérations de max-pooling successives (5×5) connectées en série, élargissant le champ récepteur sans altérer la résolution spatiale.

2.2.2 Le Neck (Fusion Multi-Échelle)

Pour détecter des objets hétérogènes (balle de ping-pong vs ballon de football), le *Neck* combine un réseau de pyramides de caractéristiques (**FPN**) et un réseau d'agrégation de chemins (**PANet**). Ce couplage permet une double circulation : propagation descendante transmettant la sémantique, et ascendante réinjectant les indices géométriques.

2.2.3 La Head (Tête de Détection Découplée et Anchor-Free)

YOLO26n intègre une approche **anchor-free**, prédisant la centralité de l'objet et les distances orthogonales par rapport aux quatre bords. La tête est **découplée** (*decoupled head*) : les branches de prédiction de classification et de régression sont séparées. Cette indépendance élimine les conflits de convergence. La couche finale projette un tenseur dimensionné pour nos $N_c = 9$ classes.

2.3 Stratégie d'Adaptation et Prétraitement du Dataset

Un défi majeur réside dans la résolution native des images du problème (`IMAGE_SIZE = 64`). L'application d'un réseau convolutif profond tel que YOLO sur 64×64 pixels poserait un problème mathématique. Avec cinq opérations de sous-échantillonnage (*strides* de 2), on obtient une réduction globale de $2^5 = 32$. Une image de 64 pixels se traduirait par une carte finale de 2×2 pixels, détruisant l'information spatiale.

Pour pallier cela sans dénaturer les annotations, le pipeline met en œuvre une mise à l'échelle :

1. **Sur-échantillonnage spatial (*Upscaling*)** : Lors du chargement par les processus d'arrière-plan (`workers=2`), chaque image est redimensionnée à 640×640 pixels (`imgsz=640`) via interpolation bilinéaire. Les coordonnées absolues des boîtes sont réindexées proportionnellement.
2. **Normalisation d'intensité** : Les valeurs de luminance RGB, codées sur $[0, 255]$, sont converties en nombres à virgule flottante et projetées dans $[0.0, 1.0]$.
3. **Génération du graphe d'étiquetage via `data.yaml`** : Le dictionnaire explicite l'arborescence des ensembles d'apprentissage et de validation.

2.4 Hyperparamètres d’Entraînement et Justifications Techniques

L’exécution de la phase d’apprentissage s’appuie sur une configuration précise transmise au framework. Le tableau 2 dresse la liste de ces hyperparamètres :

Table 2: Spécifications et rôles des hyperparamètres d’optimisation

Paramètre	Valeur	Justification technique
epochs	100	Borne supérieure des passages complets sur l’apprentissage.
device	0	Alloue l’exécution sur l’appareil CUDA d’index 0.
imgsz	640	Résolution géométrique cible forcée (Upscaling).
workers	2	Sous-processus CPU alloués au chargement des images.
cache	True	Mise en cache en RAM pour éliminer les latences I/O disque.
patience	25	Seuil d’arrêt précoce (<i>Early Stopping</i>) si pas d’amélioration du mAP.
optimizer	’auto’	Sélection automatique (AdamW ou SGD) selon les données.
batch	−1	Auto-batching : Calcule dynamiquement la taille de lot via VRAM.

Le paramètre `batch=−1` évite les erreurs *Out Of Memory*. L’algorithme teste la mémoire vidéo lors de l’initialisation et fixe la taille de lot la plus élevée possible (ex. 16, 32 ou 64).

2.5 Modélisation Mathématique des Fonctions de Perte

La convergence de YOLO26n repose sur la minimisation d’une fonction de perte globale composite $\mathcal{L}_{\text{total}}$, somme pondérée de trois objectifs distincts :

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}} \quad (2)$$

2.5.1 Perte de Régression des Boîtes Englobantes ($\mathcal{L}_{\text{box}}$)

Le réseau emploie la perte **CIoU** (*Complete Intersection over Union*), intégrant la zone de recouvrement, la distance des centres et le ratio d’aspect :

$$\mathcal{L}_{\text{box}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (3)$$

Où $\mathbf{b}$ et $\mathbf{b}^{gt}$ désignent la boîte prédite et la vérité terrain (*ground truth*). Le terme ρ est la distance euclidienne des centres, c la diagonale du plus petit rectangle englobant, et v quantifie la disparité des ratios d’aspect.

2.5.2 Perte de Classification Multiclasse ($\mathcal{L}_{\text{cls}}$)

La fonction de perte est l’entropie croisée binaire (*BCE - Binary Cross Entropy*), calculée indépendamment pour chacune des $N_c = 9$ classes :

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N_c} \sum_{k=1}^{N_c} [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)] \quad (4)$$

2.5.3 Perte de Focalisation de Distribution ($\mathcal{L}_{df}$)

La perte **DFL** (*Distribution Focal Loss*) force l'apprentissage d'une distribution discrète autour des coordonnées des bords, gérant l'incertitude géométrique :

$$\mathcal{L}_{df}(P_i, P_{i+1}) = - \left[(y_{i+1} - y^{gt}) \log(P_i) + (y^{gt} - y_i) \log(P_{i+1}) \right] \quad (5)$$

2.6 Script Logiciel d'Exécution et d'Évaluation

Le script Python listé ci-après implémente le protocole d'entraînement décrit. Il configure le modèle, lance l'optimisation, puis bascule sur la méthode de validation globale `model.val()` :

```
1 from ultralytics import YOLO
2
3 # 1. Initialisation de l'architecture algorithmique
4 model = YOLO("yolo26n.pt")
5
6 # 2. Lancement de la phase d'apprentissage supervisé
7 results = model.train(
8     data="./Projet-Apprentissage/data.yaml",
9     epochs=100,
10    device=0,
11    imgsz=640,
12    workers=2,
13    cache=True,
14    patience=25,
15    optimizer='auto',
16    batch=-1
17 )
18
19 # 3. Phase d'évaluation et de validation finale
20 results = model.val()
```

Listing 1: Script d'apprentissage YOLO26n

À l'issue de l'exécution, l'ensemble des données statistiques (courbes de *loss*, matrices de confusion, mAP_{50} , mAP_{50-95}) sont sérialisés localement dans `runs/detect/train/`, constituant le socle quantitatif pour l'analyse des résultats de la Section 3.